

CS210 Assignment02

Binary Search and Recursive Analysis

Author: Matthew Woo

Date: Sep 12, 2026

```
// AI Disclose
/**I used ChatGPT to debug.
 * I found out that countR will not work in recursive because the function is calling
 * itself
 * and it will keep setting it to 0.
 * So instead putting it into the function we should set 0 in the test function
 * and pass it into the function.
 * They also point out that I should use int& instead of int for countR
 * so it would not only pass the copy of the value.
 */
```

I also asked AI how can I connect $O(\log n)$ to $T(n)$.

Required implementation

1. Implement an iterative binary search.

```
int binarySearchIterative(vector<int> numbers, int numbersSize, int key, int& countI) {
    // Set the lowest index to 0 and the highest index to the size of numbers -1
    int low = 0;
    int high = numbersSize - 1;

    // While high is larger than or equal to low
    while (high >= low) {
        // Count comparisons for binarySearchIterative
        countI++;
        // Define mid index
        int mid = (high + low) / 2;
        // if key is larger then the numbers in the middle then set low to mid+1 to
        // look at the last half of the numbers
        if (numbers[mid] < key) {
            low = mid + 1;
        }
        // else look at the first half by setting the high value to mid-1
        else if (numbers[mid] > key) {
            high = mid - 1;
        }
        // else the key is in the middle so return mid index
        else {
            return mid;
        }
    }

    return -1; // not found
}
```

2. Implement a recursive binary search.

```
int binarySearchRecursive(vector<int> numbers, int low, int high, int key, int& countR) {
    // If low larger than high, then return -1 as no solution
    // because to use binary search numbers need to be sorted.
    if (low > high) {
        return -1;
    }

    // Define mid index in numbers
    int mid = (low + high) / 2;
    // Count how many comparisons the function did
    countR++;
    // If key is larger than the value in mid index, then check the second half of numbers
    if (numbers[mid] < key) {
        return binarySearchRecursive(numbers, mid + 1, high, key, countR);
    }
    // else check the first half if the key is smaller than the value in mid index in
    // numbers
    else if (numbers[mid] > key) {
        return binarySearchRecursive(numbers, low, mid - 1, key, countR);
    }
    // If the key is the mid value then return mid
    return mid;
}
```

3. Instrument both versions to count element comparisons.

countI in binarySearchIterative and countR in binarySearchRecursive count element comparisons.

4. Test at least five searches, including: first element, last element, middle element, missing value below the range, and missing value inside the range.

[Output](#)

5. For the recursive version, write and explain the recurrence $T(n) = T(n/2) + O(1)$ and connect it to $O(\log n)$.

$T(n)$ is the time to search through n elements,

$T(n/2)$ is the time to search through half of the list after it cuts the list in half and only looks at that half part,

$O(1)$ is the complexity for checking if the value matches the key, calculate mid, etc.

In this case, we have 8 elements. If we keep cutting it in half, it will need 3 steps to get to 1 element.

$$n = 8$$

n/2	Remaining Size	Power Form
8/2 = 4	== n/2	== $n/2^1$

n/2	Remaining Size	Power Form
4/2 = 2	== n/4	== $n/2^2$
2/2 = 1	== n/8	== $n/2^3$

It takes $\log_2(8) = 3$ steps to cut elements from 8 to 1

Therefore the time complexity is $O(\log n)$

6. Compare binary search to a linear search on the same data.

Linear Search:

```
int linearSearch(vector<int> numbers, int key, int& countLinear) {
    for (int i = 0; i < numbers.size(); ++i) {
        // Set value to current number that are looking at
        int value = numbers[i];
        // Count linear for each time the forloop statement run
        countLinear++;
        // if the value is the key that we want to find then return that index
        if (value == key) {
            return i;
        }
    }
    // If nothing was found then return -1
    return -1;
}
```

numbers = {2, 4, 7, 10, 11, 32, 45, 87}

Search for key	Binary	Linear
2	3	1
87	4	8
10	1	4
1	3	8
8	3	8

Linear search finds the key by looking at every element, and binary search looks for the key by looking at the middle value and determining which half it will look in: the lower half or the upper half. Binary search requires the list to be sorted.

If the key is at the beginning, linear search is faster than binary search to find the index, but if the value is near the middle, binary search is faster. Also, in a larger list, binary search will be faster because it cuts the half of the list.

Design Decision

There are 6 function, `binarySearchIterative`, `binarySearchRecursive`, `linearSearch`, and 3 test functions for those search method.

It only used a list of numbers and 5 test cases, including: first element, last element, middle element, missing value below the range, and missing value inside the range.

Complexity discussion

Search Method	Time Complexity	Space Complexity
Iterative Binary Search	$O(\log n)$	$O(1)$
Recursive Binary Search	$O(\log n)$	$O(\log n)$
Linear Search	$O(n)$	$O(1)$

For Iterative binary search, the time complexity is $O(\log n)$ because the range is cut in half each step, and the space complexity is $O(1)$ because it uses the same variables and doesn't use extra memory that grows with the size or make the size of the vector grow.

For Recursive binary search, the time complexity is $O(\log n)$ because the range is cut in half each recursive call; the space complexity is also $O(\log n)$ because it needs to remember every call for the recursion, so it depends on how many recursive calls it takes.

For linear search, the time complexity is $O(n)$ because it goes through n elements in the worst case; the space complexity is $O(1)$ because it doesn't take extra memory that grows with the size of the list, and it just "messes" around with the same list.

Code

[Woo Matthew Assignment02.cpp](#)

```
// AI Disclose
/**I used ChatGPT to debug.
 * I found out that countR will not work in recursive because the function is calling
 itself
 * and it will keep setting it to 0.
 * So instead putting it into the function we should set 0 in the test function
 * and pass it into the function.
 * They also point out that I should use int& instead of int for countR
 * so it would not only pass the copy of the value.
```

```

*/

#include <iostream>
#include <vector>
using namespace std;

int binarySearchIterative(vector<int> numbers, int numbersSize, int key, int& countI) {
    // Set the lowest index to 0 and the highest index to the size of numbers -1
    int low = 0;
    int high = numbersSize - 1;

    // While high is larger than or equal to low
    while (high >= low) {
        // Count comparisons for binarySearchIterative
        countI++;
        // Define mid index
        int mid = (high + low) / 2;
        // if key is larger then the numbers in the middle then set low to mid+1 to
        // look at the last half of the numbers
        if (numbers[mid] < key) {
            low = mid + 1;
        }
        // else look at the first half by setting the high value to mid-1
        else if (numbers[mid] > key) {
            high = mid - 1;
        }
        // else the key is in the middle so return mid index
        else {
            return mid;
        }
    }

    return -1; // not found
}

int binarySearchRecursive(vector<int> numbers, int low, int high, int key, int& countR) {
    // If low larger than high, then return -1 as no solution
    // because to use binary search numbers need to be sorted.
    if (low > high) {
        return -1;
    }

    //Define mid index in numbers
    int mid = (low + high) / 2;
    // Count how many comparesion the function did
    countR++;
    // If key is larger than the value in mid index, then check the second half of numbers
    if (numbers[mid] < key) {
        return binarySearchRecursive(numbers, mid + 1, high, key, countR);
    }
    // else check the first half if the key is smaller than the value in mid index in
    // numbers
    else if (numbers[mid] > key) {
        return binarySearchRecursive(numbers, low, mid - 1, key, countR);
    }
    // If the key is the mid value then return mid
    return mid;
}

int linearSearch(vector<int> numbers, int key, int& countLinear) {
    for (int i = 0; i < numbers.size(); ++i) {

```

```

        // Set value to current number that are looking at
        int value = numbers[i];
        // Count linear for each time the forloop statement run
        countLinear++;
        // if the value is the key that we want to find then return that index
        if (value == key) {
            return i;
        }
    }
    // If nothing was found then return -1
    return -1;
}

// Test Functions for iterative binary search, recursive binary search, and linear search
int testIterative(vector<int> numbers, int key) {
    int countI = 0;
    int keyIndex1 = binarySearchIterative(numbers, numbers.size(), key, countI);
    cout << "Key Index #1 (Iterative)" << endl;
    // If the function return -1, that's mean it couldn't find the key on numbers.
    if (keyIndex1 == -1) {
        cout << key << " was not found." << endl;
    }
    // else if the key was found then print out which index it found the key
    else {
        cout << "Found " << key << " at index " << keyIndex1 << "." << endl;
    }
    // print how many times binarySearchIterative compare values
    cout << "Count I: " << countI << "\n" << endl;
    return 0;
}

int testRecursive(vector<int> numbers, int low, int high, int key) {
    // set countR to 0
    int countR = 0;
    // The value that binarySearchRecursive return will store in the variable keyIndex2
    int keyIndex2 = binarySearchRecursive(numbers, low, high, key, countR);
    cout << "Key Index #2 (Recursive)" << endl;
    // If the function return -1, then print out the key was not found
    if (keyIndex2 == -1) {
        cout << key << " was not found." << endl;
    }
    // else if the key was found then print out where is the key and at which index
    else {
        cout << "Found " << key << " at index " << keyIndex2 << "." << endl;
    }
    // print the count of how many comparisons the function did
    cout << "Count R: " << countR << "\n" << endl;
    return 0;
}

int testLinear(vector<int> numbers, int key) {
    // set countLinear to 0
    int countLinear = 0;
    // The value that linearSearch return will store in the variable keyIndex3
    int keyIndex3 = linearSearch(numbers, key, countLinear);
    cout << "Key Index #3 (Linear)" << endl;
    // if the function return -1, then print the key was not found
    if (keyIndex3 == -1) {
        cout << key << " was not found." << endl;
    }
    // else if the key was found then print the index where it found the key

```

```

else {
    cout << "Found " << key << " at index " << keyIndex3 << "." << endl;
}
// print the count for linear search (how many runs it takes to find the key)
// for linear search it will go through each elements until it find the key
cout << "Count Linear: " << countLinear << endl;
return 0;
}

int main() {
    vector<int> numbers = { 2, 4, 7, 10, 11, 32, 45, 87 };
    //print out numbers
    cout << "numbers = {" ;
    for (int i = 0; i < numbers.size(); ++i) {
        cout << numbers[i];
        if (i < numbers.size() - 1) {
            cout << ", ";
        }
    }
    cout << "}\n";

    // Test Cases
    cout << "===Test Case 1, first element===\n" << endl;
    int key = 2;
    cout << "Key1: " << key << "\n";
    testIterative(numbers, key);
    testRecursive(numbers, 0, numbers.size() - 1, key);
    testLinear(numbers, key);
    cout << "\n";

    cout << "===Test Case 2, last element===\n" << endl;
    int key1 = 87;
    cout << "Key2: " << key1 << "\n";
    testIterative(numbers, key1);
    testRecursive(numbers, 0, numbers.size() - 1, key1);
    testLinear(numbers, key1);
    cout << "\n";

    cout << "===Test Case 3, middle value===\n" << endl;
    int key2 = 10;
    cout << "Key3: " << key2 << "\n";
    testIterative(numbers, key2);
    testRecursive(numbers, 0, numbers.size() - 1, key2);
    testLinear(numbers, key2);
    cout << "\n";

    cout << "===Test Case 4, missing value below the range===\n" << endl;
    int key3 = 1;
    cout << "Key4: " << key3 << "\n";
    testIterative(numbers, key3);
    testRecursive(numbers, 0, numbers.size() - 1, key3);
    testLinear(numbers, key3);
    cout << "\n";

    cout << "===Test Case 5, missing value inside the range===\n" << endl;
    int key4 = 8;
    cout << "Key5: " << key4 << "\n";
    testIterative(numbers, key4);
    testRecursive(numbers, 0, numbers.size() - 1, key4);
    testLinear(numbers, key4);
    cout << "\n";
}

```

```
    return 0;  
}
```

Output

```
numbers = {2, 4, 7, 10, 11, 32, 45, 87}  
===Test Case 1, first element===
```

```
Key1: 2  
Key Index #1 (Iterative)  
Found 2 at index 0.  
Count I: 3
```

```
Key Index #2 (Recursive)  
Found 2 at index 0.  
Count R: 3
```

```
Key Index #3 (Linear)  
Found 2 at index 0.  
Count Linear: 1
```

```
===Test Case 2, last element===
```

```
Key2: 87  
Key Index #1 (Iterative)  
Found 87 at index 7.  
Count I: 4
```

```
Key Index #2 (Recursive)  
Found 87 at index 7.  
Count R: 4
```

```
Key Index #3 (Linear)  
Found 87 at index 7.  
Count Linear: 8
```

```
===Test Case 3, middle value===
```

```
Key3: 10  
Key Index #1 (Iterative)  
Found 10 at index 3.  
Count I: 1
```

```
Key Index #2 (Recursive)  
Found 10 at index 3.  
Count R: 1
```

```
Key Index #3 (Linear)  
Found 10 at index 3.  
Count Linear: 4
```

```
===Test Case 4, missing value below the range===
```

```
Key4: 1  
Key Index #1 (Iterative)  
1 was not found.  
Count I: 3
```

```
Key Index #2 (Recursive)
```

1 was not found.
Count R: 3

Key Index #3 (Linear)
1 was not found.
Count Linear: 8

===Test Case 5, missing value inside the range===

Key5: 8
Key Index #1 (Iterative)
8 was not found.
Count I: 3

Key Index #2 (Recursive)
8 was not found.
Count R: 3

Key Index #3 (Linear)
8 was not found.
Count Linear: 8